1. List Interface ⭐ (80%)

List is an ordered collection in Java.

Meaning:

- insertion order is maintained
- duplicate values are allowed
- indexing is available (0,1,2...)
- elements can be accessed using index

Example:

[10, 20, 10, 30]

Here:

- order is maintained
- duplicate 10 is allowed

List Hierarchy

Iterable

↑

Collection

↑

List

↑ ↑ ↑

ArrayList LinkedList Vector

Creating a List

```
import java.util.*;


List<Integer> list = new ArrayList<>();
```

Important:

- Left side usually uses List
- Right side uses actual implementation

This gives flexibility.

Later you can change:

new LinkedList<>()

without changing the whole code.

Common Methods

1. add()

list.add(10);

list.add(20);

Output:

[10, 20]

---

2. add(index, element)

list.add(1, 99);

Output:

[10, 99, 20]

---

3. get(index)

System.out.println(list.get(1));

Output:

99

---

4. set(index, element)

Replaces value.

list.set(1, 50);

Output:

[10, 50, 20]

---

5. remove()

Remove by index

list.remove(1);

Remove by object

list.remove(Integer.valueOf(10));

Important:

list.remove(1);

In Integer list, it removes by index.

---

6. size()

list.size();

---

7. contains()

list.contains(20);

Returns:

true

---

8. clear()

Deletes all elements.

list.clear();

Traversing a List

for loop

```
for (int i = 0; i < list.size(); i++) {
   System.out.println(list.get(i));
}
```

---

enhanced for loop

```
for (Integer num : list) {
   System.out.println(num);
}
```

---

iterator

```
Iterator<Integer> it = list.iterator();


while(it.hasNext()) {
```

```
    System.out.println(it.next());
}
```

Full Example

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        List<String> fruits = new ArrayList<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");

        System.out.println(fruits);

        fruits.add(1, "Orange");

        System.out.println(fruits);

        fruits.set(2, "Grapes");

        System.out.println(fruits);

        System.out.println(fruits.get(1));

        fruits.remove("Apple");

        System.out.println(fruits);
```

```
        System.out.println(fruits.contains("Mango"));

        for(String fruit : fruits) {
            System.out.println(fruit);
        }
    }
}
```

Output

[Apple, Banana, Mango]

[Apple, Orange, Banana, Mango]

[Apple, Orange, Grapes, Mango]

Orange

[Orange, Grapes, Mango]

true

Orange

Grapes

Mango

Array vs List

Feature	Array	List
Size	fixed	dynamic
Primitive support	yes	wrapper classes
Built-in methods	less	many
Resizable	no	yes

---

2. ArrayList ⭐🔥 (95%)

Most used collection in Java.

Internal Working

ArrayList internally uses:

dynamic array

Imagine

[10, 20, 30, _, _, _]

When full:

- a bigger array is created
- old elements are copied

Because of this:

- random access is FAST
- middle insertion/removal is SLOW

Syntax

ArrayList<Integer> list = new ArrayList<>();

Capacity vs Size ⭐

Size

Actual number of elements.

Capacity

Internal array size.

Example:

ArrayList<Integer> list = new ArrayList<>();

Default capacity:

10

But size:

0

Resizing

When capacity becomes full:

Old:

10

New:

10 + (10/2) = 15

Approx:

1.5x grow

Time Complexity ⭐

Operation	Complexity
add() at end	O(1) average
get()	O(1)
set()	O(1)
remove() end	O(1)
insert middle	O(n)
remove middle	O(n)

Why is get() Fast?

Because array indexing is used.

address = base + index * size

Direct memory jump.

ArrayList Example

```
import java.util.*;


public class Main {
   public static void main(String[] args) {


      ArrayList<Integer> list = new ArrayList<>();


      list.add(10);
      list.add(20);
```

```
        list.add(30);

        System.out.println(list);

        System.out.println(list.get(1));

        list.set(1, 99);

        System.out.println(list);

        list.remove(0);

        System.out.println(list);
    }
}
```

Output

```
[10, 20, 30]
20
[10, 99, 30]
[99, 30]
```

Important Constructors

Default

```
ArrayList<Integer> list = new ArrayList<>();
```

Custom capacity

```
ArrayList<Integer> list = new ArrayList<>(100);
```

Useful when data size is known.

Less resizing happens.

Copy another collection

ArrayList<Integer> list2 = new ArrayList<>(list1);

Primitive Problem ⭐

Wrong:

ArrayList<int> list

Correct:

ArrayList<Integer> list

Because generics do not support primitives.

Wrapper classes are used.

Primitive	Wrapper
int	Integer
char	Character
double	Double

Autoboxing / Unboxing

list.add(10);

Actually becomes:

Integer.valueOf(10)

Automatically handled by Java.

ArrayList Traversal

Normal loop

```
for(int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}
```

---

for-each

```
for(Integer val : list) {
    System.out.println(val);
```

}

---

iterator

Iterator<Integer> it = list.iterator();

while(it.hasNext()) {

System.out.println(it.next());

}

Sorting ⭐

Collections.sort(list);

Example:

ArrayList<Integer> list = new ArrayList<>();

list.add(40);

list.add(10);

list.add(30);

Collections.sort(list);

System.out.println(list);

Output:

[10, 30, 40]

Reverse

Collections.reverse(list);

Convert Array → ArrayList

Integer[] arr = {1,2,3};

List<Integer> list = Arrays.asList(arr);

Important:
asList() returns fixed-size list.

list.add(4);

Will throw error.

Convert ArrayList → Array

Object[] arr = list.toArray();

Common Interview Questions ⭐

ArrayList vs Array

Array	**ArrayList**
fixed size	dynamic size
primitive allowed	wrapper only
faster	slightly slower
fewer methods	many methods

ArrayList vs LinkedList

ArrayList	**LinkedList**
fast random access	slow random access
slow middle insertion	faster insertion
less memory	more memory
mostly used	niche use

When to Use ArrayList ⭐

Use when:

- frequent reading/access
- searching by index
- dynamic data storage

Avoid when:

- frequent middle insertion/removal

Common Mistakes ⭐

1. remove() confusion

list.remove(1);

Removes by index.

---

2. Using primitive

Wrong:

ArrayList<int>

---

3. Loop condition mistake

Wrong:

i <= list.size()

Correct:

i < list.size()

---

3. LinkedList ⭐ (40%)

Basic understanding is enough.

Internal Structure

[data | next]

Actually Java uses doubly linked list:

prev | data | next

Structure

10 ↔ 20 ↔ 30

Each node stores:

- previous address
- data
- next address

Syntax

LinkedList<Integer> list = new LinkedList<>();

Why Use LinkedList?

Middle insertion/removal is efficient.

Because shifting is not needed.

Problem

Index access is slow.

list.get(5000)

Traversal happens internally.

Time Complexity

Operation	Complexity
get()	O(n)
addFirst()	O(1)
addLast()	O(1)
removeFirst()	O(1)

Example

```
import java.util.*;


public class Main {
   public static void main(String[] args) {


      LinkedList<Integer> list = new LinkedList<>();


      list.add(10);
      list.add(20);
```

```
        list.addFirst(5);
        list.addLast(30);

        System.out.println(list);

        list.removeFirst();

        System.out.println(list);
    }
}
```

Output:

[5, 10, 20, 30]

[10, 20, 30]

ArrayList vs LinkedList

ArrayList

- dynamic array
- fast access
- slow middle insertion

LinkedList

- node based
- slow access
- better insertion/removal

Real World

In most cases:

ArrayList

is preferred.

---

4. Vector (5%)

Old legacy class.

Vector<Integer> v = new Vector<>();

Difference:

- synchronized
- slower

Modern Java mostly prefers:

ArrayList

Enough for now.

---

5. Stack (10%)

LIFO:

Last In First Out

Example:

Plate stack

Methods

push()

pop()

peek()

Example

Stack<Integer> st = new Stack<>();

st.push(10);

st.push(20);

System.out.println(st.pop());

System.out.println(st.peek());

Output:

20

10

Modern Java usually prefers:

Deque

instead of Stack.

---

6. CopyOnWriteArrayList (20%)

Thread-safe list.

Package:

java.util.concurrent

Concept

Whenever modification happens:

- original array gets copied
- changes happen on new array

Because of this:

- reading is safe
- iteration is safe
- writing is expensive

Syntax

```
CopyOnWriteArrayList<Integer> list =
    new CopyOnWriteArrayList<>();
```

Why Needed?

Normal ArrayList can throw:

ConcurrentModificationException

during iteration.

Example

```
import java.util.concurrent.CopyOnWriteArrayList;

public class Main {
```

```
    public static void main(String[] args) {

        CopyOnWriteArrayList<Integer> list =
                new CopyOnWriteArrayList<>();

        list.add(1);
        list.add(2);

        for(Integer num : list) {

            System.out.println(num);

            list.add(100);
        }

        System.out.println(list);
    }
}
```

Output

1

2

[1, 2, 100, 100]

No exception occurs.

Final Practical Priority ⭐

1. ArrayList ← MASTER THIS
2. List Interface
3. LinkedList basics
4. Stack basic concept
5. Vector awareness

6. CopyOnWriteArrayList basic idea